

Stan na 31 lipca 2026 · aplikacja 1.0.2 · adresy i porty odczytane z działających kontenerów, plików compose i definicji przepływów w n8n

Stan na 31 lipca 2026 · aplikacja 1.0.2 · adresy i porty odczytane z działających kontenerów, plików compose i definicji przepływów w n8n

```

graph LR
    A["Repozytorium  
github.com/Marcin-CCC/zco-edm"] --> B["GitHub Actions  
job „build” · ubuntu-latest"]
    B --> C["ghcr.io  
backend:latest · frontend:latest"]
    C --> D["Runner self-hosted na Sparku  
usługa systemd: spark-zco-edm"]
    D --> E["docker compose --profile spark  
up -d backend frontend → nowe kontenery"]
  
```

tryb deweloperski: ta sama baza danych (5433)

192.168.1.17 · Docker Desktop · praca deweloperska

192.168.1.17 · Docker Desktop · praca deweloperska

localhost:3002 — interfejs
localhost:8001/docs — API

```
zco-edm-final-frontend-1 · Next.js
3002 → 3000
proxy /api → backend:8001
```

HTTP

```
zco-edm-final-backend-1 · FastAPI
8001 → 8000
kod montowany z dysku (backend/app)
konfiguracja: backend/.env.dev
```

Edge headless: zrzuty ekranu i PDF
docs/instrukcje, docs/prezentacja

kod, testy, pomiary
wypchnięcie na master = wdrożenie

192.168.1.34 · 128 GB pamięci zunifikowanej · usługi AI, dane, wdrożenie

192.168.1.34 · 128 GB pamięci zunifikowanej · usługi AI, dane, wdrożenie

obraz z ghcr
3000 → 3000
wersja dla klie

obraz z ghcr
8083 → 8000
ten sam kod c

PostgreSQL 15
5433 → 5432
konto, pliki, rozmowy

odesłanie: status pliku i lista źródeł odpowiedzi

webhook parsowania (tryb deweloperski)

n8n woła usługi parsowania (HTTP)

Page 10 of 10

edm-docling
8085 → 5001
/v1/convert/file
tekst i tabele

```
zco-document-rasterizer
8084
/rasterize (PDF → obrazy)
/convert-to-docx (ODT → DOCX)
```

excel-parser
8086
/process-excel
arkusze XLSX

wektoryzacja pytań i fragmentów, zapis do bazy wektorowej

Page 10 of 10

qwen3vl-vllm	8002	Qwen/Qwen3-VL-30B-A3B-Instruct	odpowiedzi czatu, rozpoznawanie dokumentów, streszczenia
--------------	------	--------------------------------	--

ollama
11434
bge-m3:latest
zamiana tekstu na wektory: fragmenty, pytania, streszczenia

qdrant
6333
chi_camp_2026 — fragmenty
chi_camp_2026_streszczenia — opisy

kopiowanie wgranych plików przez SSH do /data/shared_docs

backend wprost do modeli i wektorów (streszczenia, rozpoznawanie, wyszukiwanie)

192.168.1.34:3000 — aplikacja
ruch nie opuszcza sieci lokalnej

— wywołania aplikacji (HTTP, SQL)

- - - połączenia zestawiane tylko w trybie deweloperskim

— przepływy w n8n: parsowanie i czat

— budowanie i wdrożenie (CI/CD)

Jak to czytać

Rysunek pokazuje DWA tryby pracy tej samej aplikacji. **Tryb deweloperski**: kod działa w kontenerach na komputerze lokalnym, ale wszystkie dane i modele bierze ze Sparka — dlatego szare przerywane linie przecinają granicę stref. **Tryb wdrożony**: identyczny kod działa w kontenerach na Sparku i nie sięga poza tę maszynę.

Co z tego wynika w praktyce

- **Jedna baza danych i jedna baza wektorowa dla obu trybów.** Praca deweloperska widzi te same konta, pliki i rozmowy co wersja demonstracyjna — zmiana danych lokalnie jest zmianą u klienta. To najważniejsze ograniczenie tego układu.
- **n8n zawsze stoi na Sparku.** Lokalny n8n na porcie 5678 należy do innego projektu i nie uczestniczy w tym przepływie.
- **Odesłania z n8n trafiają pod inny adres w każdym trybie** (`BACKEND_CALLBACK_URL`): deweloperski `192.168.1.17:8001` , wdrożony `192.168.1.34:8083` . To jedyne miejsce, w którym n8n musi wiedzieć, który backend go zawołał.
- **Pliki wgrane lokalnie są kopiowane na Sparka przez SSH**, bo n8n czyta je z wolumenu `/data/shared_docs` . Bez tego kroku parsowanie w trybie deweloperskim nie ma czego czytać.
- **Całe wdrożenie dzieje się na Sparku.** Runner GitHub Actions działa tam jako usługa systemd: buduje obrazy natywnie pod ARM64, wypycha do ghcr.io i podnosi kontenery.

Dwa przepływy przez n8n

- **Parsowanie dokumentu** — backend wysyła webhook po wgraniu pliku. n8n rozdziela pracę według formatu: PDF przez rasteryzator i model widzenia, DOCX i ODT przez Docling (ODT najpierw zamieniany na DOCX), XLSX przez excel-parser. Fragmenty trafiają do Qdranta, a status wraca do backendu.
- **Pytanie w czacie** — backend woła webhook strumieniowy. n8n pobiera 15 fragmentów z Qdranta (z filtrem uprawnień przygotowanym przez backend), odrzuca te poniżej progu trafności 0,50, buduje kontekst z etykietami źródeł, pyta model i odsyła listę użytych dokumentów do backendu.

Czego na rysunku nie ma

Pominałem kontenery innych projektów działających na obu maszynach (iwound-lab, appsmith, comfyui, open-webui, lokalny n8n) — nie należą do tego środowiska i tylko zaciemniałyby obraz. Pominąłem też sekrety: nagłówki uwierzytelniający webhooks i hasła baz są w plikach konfiguracyjnych, nie na schemacie.

Skąd wzięte dane

`docker ps` na obu maszynach, zmienne środowiskowe kontenerów, `docker-compose*.yaml` , `backend/.env.dev` , definicje obu przepływów odczytane z API n8n oraz `systemctl` dla runnera. Stan na 31 lipca 2026, aplikacja 1.0.2.